



CSV Source as an Argument Provider

JUnit offers an additional annotation for supplying arguments: `@CsvSource`. It **replaces** `@MethodSource` when you want **simpler inline data** and makes **parameterized tests shorter** and **easier to read** for **simple data**.



by Rodrigo Martins Pagliares

Last update: 11/27/2025

Example – Subtraction Test

01

Replace `@MethodSource` with `@CsvSource`

02

Provide sets like "113, 13, 100" for minuend, subtrahend, and expected result

03

Add more sets such as "15, 15, 0" or "23, 24, -1"

04

Test runs once per entry, supplying the mapped values automatically



@CsvSource as an alternative to @MethodSource

```
@CsvSource({"113, 13, 100"})
```

Calculator.java

CalculatorTest.java

```
15      public class CalculatorTest {
60          @DisplayName("Test integer subtraction (minuend, subtrahend, expectedResult)")
61          @ParameterizedTest
62          // @MethodSource("generateInputParametersForIntegerSubtractionTestMethod")
63          @CsvSource({"113, 13, 100"})
64          public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturn0
65              (int minuend, int subtrahend, int expectedResult) {
66
67              System.out.println(minuend + "-" + subtrahend + " = " + expectedResult);
68              Calculator calculator = new Calculator();
69
70              int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
71              assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
72                  " did not yield " + expectedResult);
73      }
```

Using @CsvSource



- Each **argument set** is enclosed in `{}` and **wrapped in double quotes**.
- **Values** inside a set are **separated by commas**:
"113, 13, 100".
- **JUnit** maps **each value** to the **test method's parameters in order**.
- **Multiple sets** can be **added, each separated by a comma**.

@CsvSource as an alternative to @MethodSource (contd.)

Multiple input/output combination

The screenshot shows an IDE window with the following content:

```
15 public class CalculatorTest {  
62 // @MethodSource("generateInputParametersForIntegerSubtractionTestMethod")  
63 @CsvSource({  
64     "113, 13, 100",  
65     "15, 15, 0",  
66     "23, 24, -1",  
67     "0, 0, 0",  
68     "0, 13, -13"  
69 })
```

Below the code editor, the Run window displays the following execution results:

- Run: CalculatorTest.testPerformIntegerSubtraction_When...
- Test operations in Calculator class (org.example): 33 ms
- Test integer subtraction (minuend, subtrahend, expectedResult): 33 ms
 - [1] "113", "13", "100": 33 ms
 - [2] "15", "15", "0"
 - [3] "23", "24", "-1"
 - [4] "0", "0", "0"
 - [5] "0", "13", "-13"
- 5 tests passed 5 tests total, 33 ms
- Executing @BeforeEach method
- 0-13 = -13
- Executing @AfterEach method

The same test method run **5 times!**

Removing Unused @MethodSource Methods

- When switching to `@CsvSource`, the **provider method** is **no longer needed**.
- You can **delete** or **comment out** the **old static method**.
- This **keeps** your **test class cleaner** and **easier to maintain**.
- Only the **annotation controls** all **input variations**.



@CsvSource as an alternative to @MethodSource (contd.)

Not needed anymore!

```
81      //      private static Stream<Arguments> generateInputParametersForIntegerSubtractionTestMethod() {
82      //          return Stream.of(
83      //              Arguments.of(113, 13, 100),
84      //              Arguments.of(15, 15, 0),
85      //              Arguments.of(23, 24, -1),
86      //              Arguments.of(0, 0, 0),
87      //              Arguments.of(0, 13, -13)
88      //          );
89      //      }
```

String Input with @CsvSource

@CsvSource also supports strings, not only numbers

Strings must be inside double quotes, e.g., "January", "May"

Empty strings are represented as "" (two single quotes)

null is represented by leaving the entry blank after the comma



CSV Source Flexibility



- **Useful** when **input** is **simple** and **fits cleanly** in **one line**.
- **Great** for testing **string combinations** or **numeric variations**.
- Helps **reduce boilerplate** by **avoiding external data provider methods**.
- Works **smoothly** with **parameterized test** signatures.



Conclusion

@CsvSource provides an easy inline alternative for parameterized tests

Ideal for simple numeric or string inputs

Reduces the need for extra methods while keeping tests readable

Supports multiple data formats including empty and null values



Open Floor Discussion

- In which cases would you prefer @MethodSource over @CsvSource?
- How does readability influence your choice of argument provider?
- Should complex input scenarios still be handled with external providers?